

```
In [2]: import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Step 1: Load and prepare the wine dataset
try:
    red = pd.read_csv("winequality-red.csv", sep=";")
    white = pd.read_csv("winequality-white.csv", sep=";")
except FileNotFoundError:
    print("Wine data files not found. Please put them in the working folder.")
    exit()

wine = pd.concat([red, white], axis=0).reset_index(drop=True)

X_raw = wine.drop("quality", axis=1)
y_raw = (wine["quality"] > 6).astype(int)    # 1 = good, 0 = bad

# min-max normalization
def normalize(df):
    return (df - df.min()) / (df.max() - df.min())

X_norm = normalize(X_raw)
X_norm["bias"] = 1    # add bias column

X = X_norm.to_numpy()
y = y_raw.to_numpy()

print("Data loaded and normalized.")

# Step 2: Logistic Regression from scratch
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

class LogisticScratch:
    def __init__(self, lr=0.1, epochs=50000):
        self.lr = lr
        self.epochs = epochs
        self.W = None

    def fit(self, X, y):
        m, n = X.shape
        self.W = np.random.randn(n, 1) * 0.01

        for _ in range(self.epochs):
            z = X @ self.W
            preds = sigmoid(z)
```

```

        grad = (X.T @ (preds - y.reshape(-1, 1))) / m
        self.W -= self.lr * grad

    def predict_prob(self, X):
        return sigmoid(X @ self.W)

    def predict(self, X):
        return (self.predict_prob(X) >= 0.5).astype(int)

print("Custom logistic regression class ready.")

# Step 3: Metrics
def compute_metrics(y_true, y_pred):
    y_true = y_true.flatten()
    y_pred = y_pred.flatten()

    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))

    acc = np.mean(y_true == y_pred)
    prec = tp / (tp + fp) if (tp + fp) > 0 else 0
    rec = tp / (tp + fn) if (tp + fn) > 0 else 0
    return acc, prec, rec

# Step 4: Manual 3-Fold Cross Validation
k = 3
folds = [[] for _ in range(k)]
class_indices = [np.where(y == c)[0] for c in np.unique(y)]

for idx in class_indices:
    np.random.shuffle(idx)
    for i, j in enumerate(idx):
        folds[i % k].append(j)

scratch_results = {"acc": [], "prec": [], "rec": []}
sk_results = {"acc": [], "prec": [], "rec": []}

sk_model = LogisticRegression(solver="saga", penalty=None, max_iter=10000)

for fold in range(k):
    print(f"\nFold {fold+1}")

    test_idx = folds[fold]
    train_idx = [j for f in range(k) if f != fold for j in folds[f]]

    X_train, y_train = X[train_idx], y[train_idx]
    X_test, y_test = X[test_idx], y[test_idx]

```

```

# Scratch model
my_model = LogisticScratch()
my_model.fit(X_train, y_train)
y_hat = my_model.predict(X_test)

a, p, r = compute_metrics(y_test, y_hat)
scratch_results["acc"].append(a)
scratch_results["prec"].append(p)
scratch_results["rec"].append(r)
print(f"Scratch -> Acc: {a:.4f}, Prec: {p:.4f}, Rec: {r:.4f}")

# Sklearn model
sk_model.fit(X_train, y_train)
y_hat2 = sk_model.predict(X_test)

a2, p2, r2 = compute_metrics(y_test, y_hat2)
sk_results["acc"].append(a2)
sk_results["prec"].append(p2)
sk_results["rec"].append(r2)
print(f"Sklearn -> Acc: {a2:.4f}, Prec: {p2:.4f}, Rec: {r2:.4f}")

print("\nAverage Results over 3 folds:")
print("Scratch Logistic Regression:")
print(f" Accuracy={np.mean(scratch_results['acc']):.4f}, Precision={np.mean(scratch_results['prec']):.4f}, Recall={np.mean(scratch_results['rec']):.4f}")
print("\nSklearn Logistic Regression:")
print(f" Accuracy={np.mean(sk_results['acc']):.4f}, Precision={np.mean(sk_results['prec']):.4f}, Recall={np.mean(sk_results['rec']):.4f}")

```

```
Data loaded and normalized.  
Custom logistic regression class ready.
```

```
Fold 1  
Scratch -> Acc: 0.8273, Prec: 0.6327, Rec: 0.2911  
Sklearn -> Acc: 0.8269, Prec: 0.6232, Rec: 0.3028
```

```
Fold 2  
Scratch -> Acc: 0.8107, Prec: 0.5513, Rec: 0.2019  
Sklearn -> Acc: 0.8112, Prec: 0.5509, Rec: 0.2160
```

```
Fold 3  
Scratch -> Acc: 0.8176, Prec: 0.5781, Rec: 0.2612  
Sklearn -> Acc: 0.8129, Prec: 0.5467, Rec: 0.2753
```

```
Average Results over 3 folds:  
Scratch Logistic Regression:  
  Accuracy=0.8185, Precision=0.5874, Recall=0.2514
```

```
Sklearn Logistic Regression:  
  Accuracy=0.8170, Precision=0.5736, Recall=0.2647
```

In []: